

Discrete
and
Distributed Word Representation

Content

- Discrete Text representation
 - N-grams model
- Introduction to Distributed word representation
 - Word2Vec
 - ✓ Skip Gram model
 - ✓ Continuous Bag Of Words model

N-grams

N-gram language model predicts the probability of the next word given (n-1) previous words

n=1
Unigram

n=2
Bigram

n=3
Trigram

Example:

"I love natural language processing"

Unigrams (n=1):

["I", "love", "natural", "language", "processing"]

Bigrams (n=2):

["I love", "love natural", "natural language", "language processing"]

Trigrams (n=3):

["I love natural", "love natural language", "natural language processing"]

Natural language is not a collection of words

The cat sat on the mat
The mat sat on the cat
The sat mat on the cat
Sat the mat on the cat



Doesn't make a meaningful sentence

Language modeling assigns probabilities to a sequence of words

The cat sat on the mat (High probability score).

The sat mat on the cat (Low probability score).

Language Modeling using Markov's Assumption

Example:

The cat sat on the mat.

First-order Markov's Assumption

This is essentially a **bigram model**.

It assumes the probability of the next word depends **only on the previous one**:

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-1})$$

➤ **Example:**

We have bigram counts from the training corpus.

- $P(\text{cat} | \text{the})$
- $P(\text{sat} | \text{cat})$
- $P(\text{on} | \text{sat})$
- $P(\text{the} | \text{on})$
- $P(\text{mat} | \text{the})$

Second-order Markov Assumption

This is a **trigram model**.

It assumes the probability of the next word depends on the **previous two words**:

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-2}, w_{t-1})$$

From the sentence we can extract trigrams:

- $P(\text{sat} | \text{the}, \text{cat})$
- $P(\text{on} | \text{cat}, \text{sat})$
- $P(\text{the} | \text{sat}, \text{on})$
- $P(\text{mat} | \text{on}, \text{the})$

Example:

The cat sat on the mat.

Now the model uses the last **two words** to predict the next one.

This will be more accurate as more words are taken into context than in first order Markov's assumption

k -th order Markov assumption

The probability of the next word depends on the **previous k words**:

$$P(w_t \mid w_1^{t-1}) \approx P(w_t \mid w_{t-k}, \dots, w_{t-1})$$

For example, with **$k=4$** (a 5-gram model), after seeing:

“the cat sat on”

The model uses those four words to predict the fifth word.

Limitations

The **animal** did not cross the **road** as it was too {**wide**/**tired**}.

Capturing long term dependency depends on the value of k chosen.

We need a language model which will considers all kinds of features of the context which might be useful in predicting a word.

Distributed Text Representation

It's a way of representing **words (or larger text units)** as **dense vectors** in a continuous space, instead of as sparse indicators (like one-hot vectors). These are called word embeddings.

Associate a real-valued d-dimensional distributed feature vector with each word in the vocabulary

Properties of these vectors

- These vectors are d dimensional where $d \ll |V|$, v being the size of the vocabulary
- Similar words occur together and have similar context.

$$\text{➤ } V_{\text{rose}} = \begin{bmatrix} 1 \cdot 6 \\ 2 \\ -5 \\ 4 \end{bmatrix} \quad V_{\text{marigold}} = \begin{bmatrix} 1 \cdot 4 \\ 2.1 \\ -5.2 \\ 4.3 \end{bmatrix} \quad V_{\text{cat}} = \begin{bmatrix} 7 \cdot 4 \\ -0.1 \\ -1.1 \\ 0.3 \end{bmatrix}$$

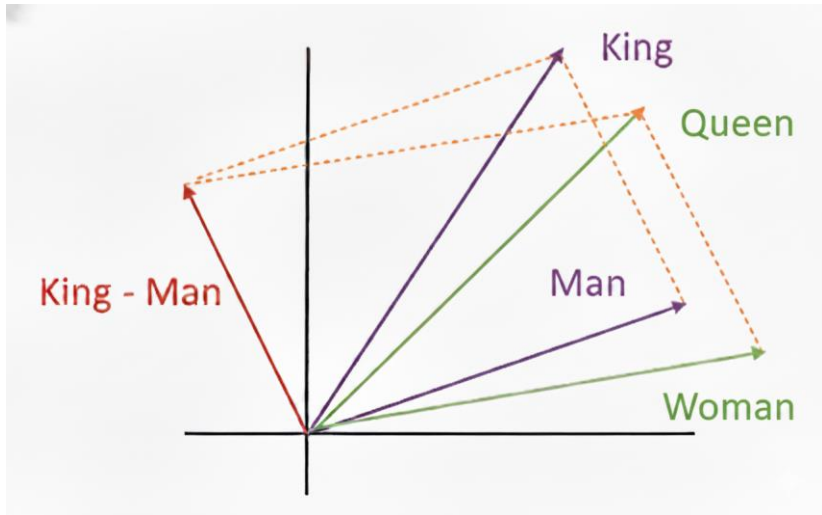
You shall know a word by the company it keeps" – J.R. Firth(1957)

Distributed Text Representation

Core idea:

Similar words tend to occur together and will have similar context

Can the Calculations using these vectors capture semantic association?



$$V_{king} - V_{man} = V_{Queen} - V_{woman}$$

“royalty” and “human” are preserved
only the gender attribute changes

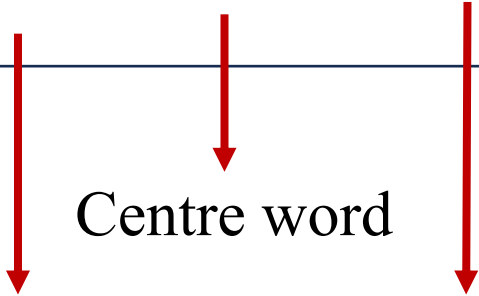
$$V_{king} - V_{man} + V_{woman} = V_{Queen}$$

Word2Vec

- ✓ It is a technique for converting words into numerical vectors (word embeddings) that capture semantic relationships, meaning words with similar contexts have similar vectors
- ✓ It's a combination of two techniques
 - Skip Gram Model
 - Continuous bag of words.
- ✓ Both of these techniques are shallow neural networks that map words to a target variable which is also a word(s).
- ✓ Both of these techniques learn weights of the neural network which act as word vector representation.

Centre word and Context word in NLP

Students enjoy reading books in the library



Window size=1

Students enjoy reading books in the library

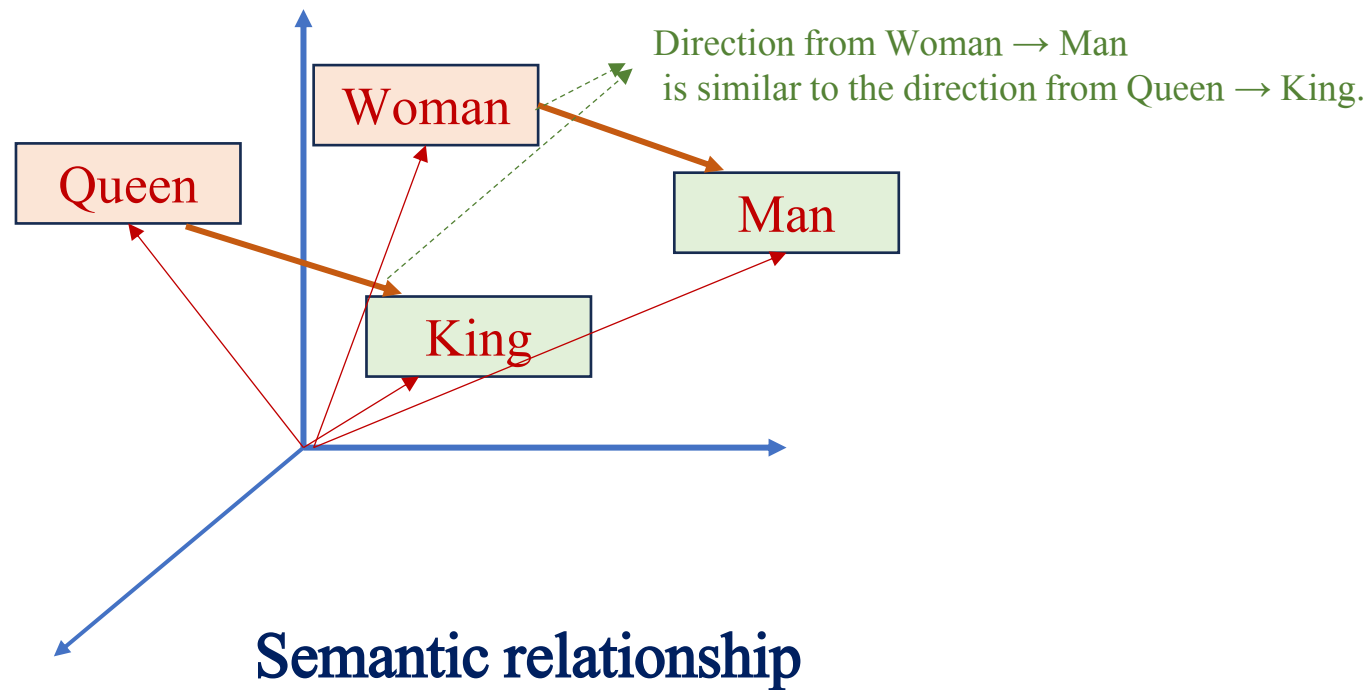
Window size=2

Word2Vec Word Embeddings

Example:

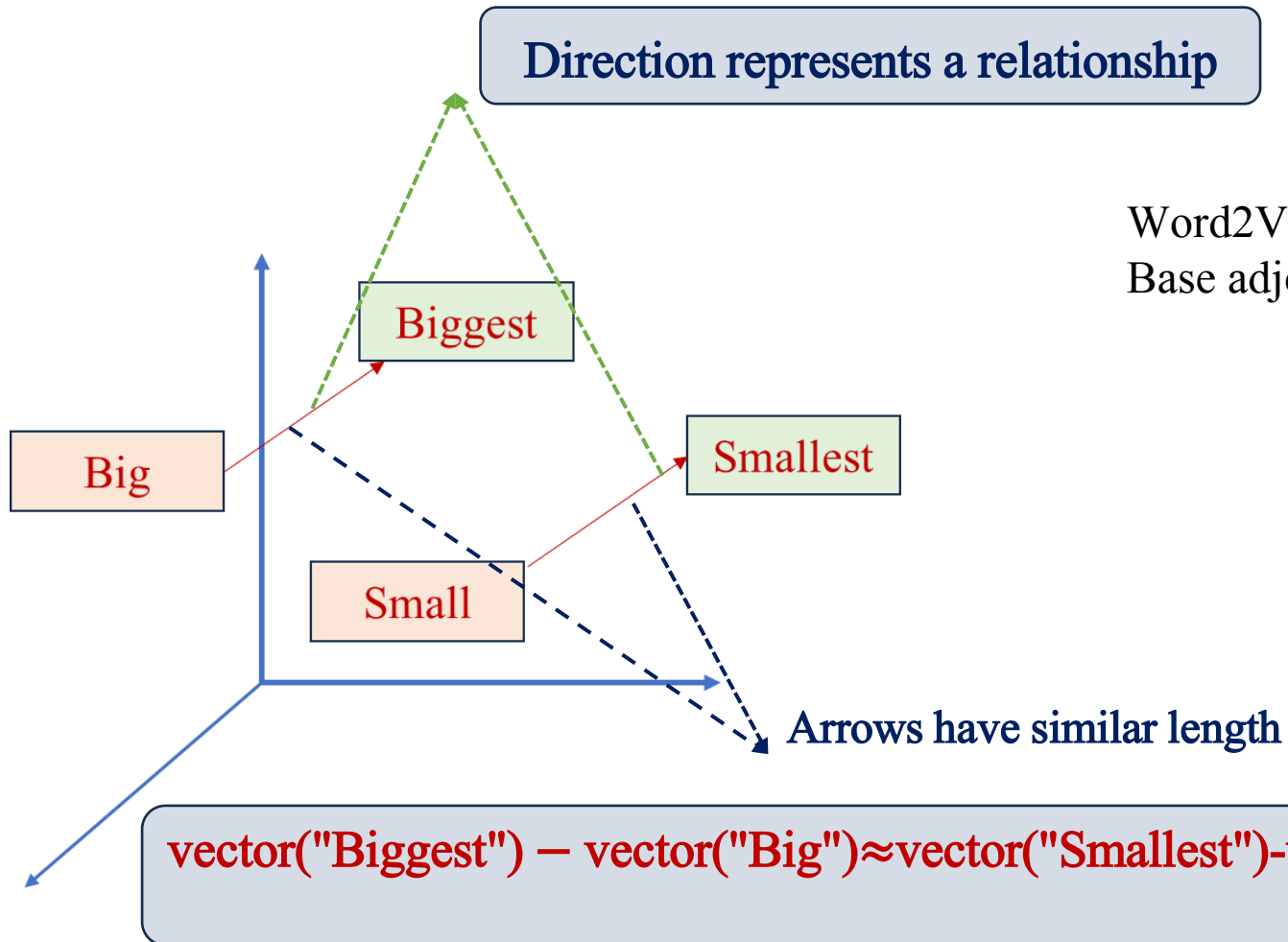
- The **king** and the **queen** rule the kingdom.
- A **man** and a **woman** lead their people

Places them in **similar contexts** (rule, lead, people).



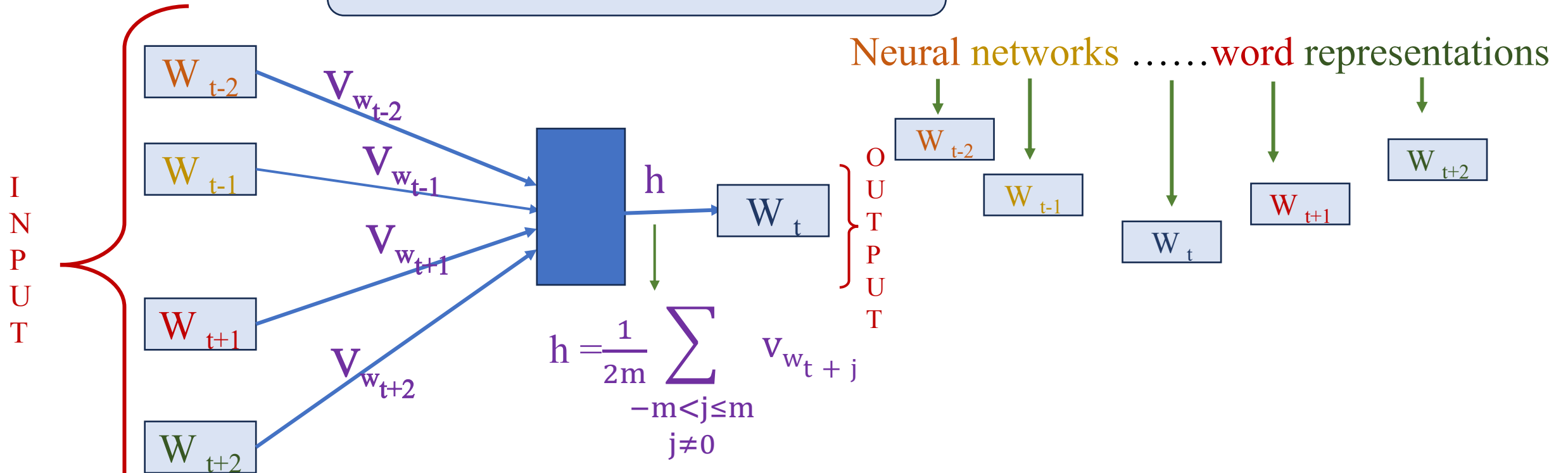
Word2Vec Word Embeddings

Big, Small, Biggest, Smallest are shown as points in that space



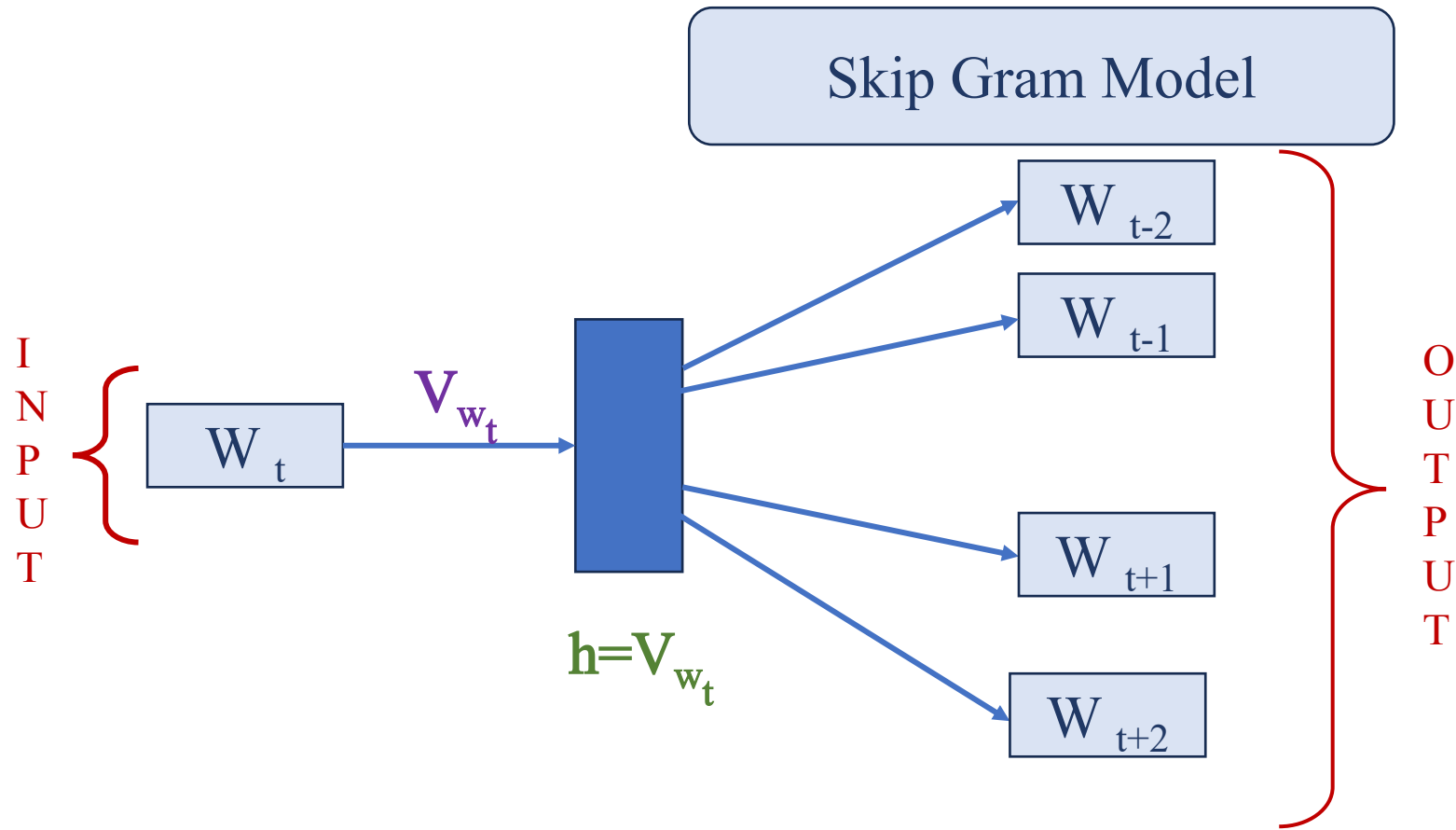
Prediction of Centre Word from Context Word(s)

Continuous Bag of Words (CBOW)



$$P(W_t = i | \text{Context words}) = \frac{\exp(h^T v_i')}{\sum_{w \in V} \exp(h^T v_w')} \rightarrow v_i' \text{ is the output embedding of word } i.$$

Prediction of Context Word(s) from Centre Word



$$P(W_{t+j} = i | W_t) = \frac{\exp(h^T v'_i)}{\sum_{w \in V} \exp(h^T v'_w)}$$

Each word in the vocabulary has an output embedding v'_w .

Summary

Thank you

Next Session

Word embeddings